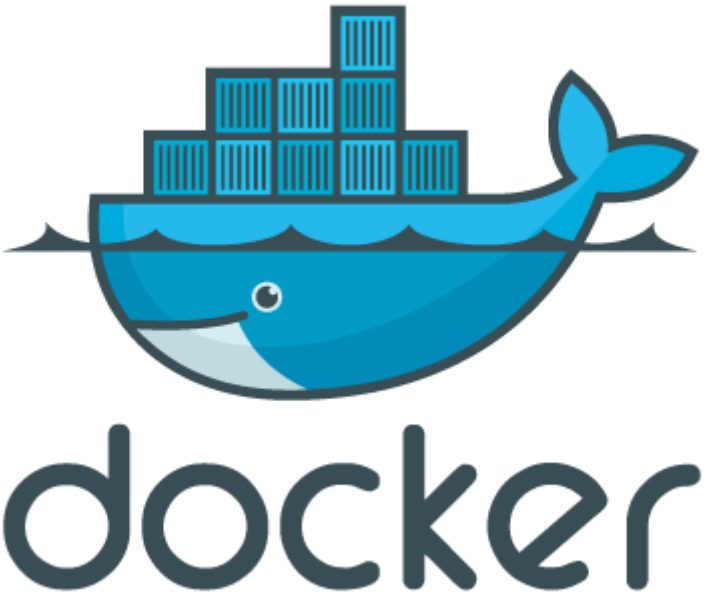
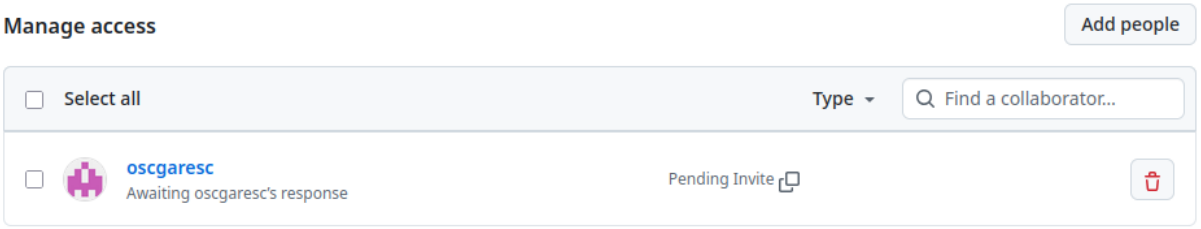
UD01 - Activitat Avaluable

ACTIVITAT 01 2
ACTIVITAT 02 2
1. Cas pràctic instal·lació de Docker de la unitat 2 2
2. Cas pràctic bàsic de comandos amb Docker. 4
5. Cas pràctic de Docker Compose 1 de la unitat 6 5
ACTIVITAT 03 7



ACTIVITAT 01

Crea un compte (si no el tens) en https://github.com . Una vegada creat, fes un repositori privat al qual podràs cridar “SistemesGestioEmpresarial” o un nom similar. Hauràs de compartir aquest repositori amb l’usuari de github “oscgaresc”.



ACTIVITAT 02

1. Cas pràctic instal·lació de Docker de la unitat 2

Actualizaremos los repositorios y instalaremos las versiones más recientes con apt update y upgrade

```
nacho@nacho-virtualbox:~/Desktop$ sudo apt update
[sudo] password for nacho:
Des:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126
kB]
Des:2 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Des:3 http://security.ubuntu.com/ubuntu noble-security/main amd64 Pack
ages [1.155 kB]
```

```
nacho@nacho-virtualbox:~/Desktop$ sudo apt upgrade
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Calculando la actualización... Hecho
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  libqt5webengine-data vlc-plugin-qt libvlc5 qt5-qmake-bin
```

Lo siguiente será instalar los paquetes necesarios, además de descargar la clave GPG del repositorio de Docker CE e incluirla

```
nacho@nacho-virtualbox:~/Desktop$ sudo apt-get install ca-certificates
 curl
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libcurl3t64-gnutls libcurl4t64
Se actualizarán los siguientes paquetes:
  ca-certificates curl libcurl3t64-gnutls libcurl4t64
```

```
nacho@nacho-virtualbox:~/Desktop$ sudo install -m 0755 -d /etc/apt/keyrings
nacho@nacho-virtualbox:~/Desktop$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyring
s/docker.asc
nacho@nacho-virtualbox:~/Desktop$ sudo chmod a+r /etc/apt/keyrings/docker.asc
```

A continuación añadiremos el repositorio de Docker CE como fuente para instalación de paquetes.

```
nacho@nacho-virtualbox:~/Desktop$ echo   "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docke
r.asc] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" |   sudo tee /etc/apt/sources.list.
d/docker.list > /dev/null
```

Por último, una vez tenemos el repositorio oficial de Docker, solo nos queda actualizar los paquetes e instalar su última versión

```
nacho@nacho-virtualbox:~/Desktop$ sudo apt update
Des:1 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Obj:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:3 http://archive.ubuntu.com/ubuntu noble InRelease
Des:4 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [73,4 kB]
```

```
nacho@nacho-virtualbox:~/Desktop$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin dock
er-compose-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  docker-ce-rootless-extras pigz
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin
  pigz
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 102 MB de archivos.
Se utilizarán 389 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
```

Para comprobar que se ha instalado correctamente ejecutaremos docker version:

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker version
Client: Docker Engine - Community
 Version:           29.9.3
 API version:       1.56
 Go version:        go1.26.10
 Git commit:        5457199
 Built:             Tue Nov  3 21:41:31 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.9.3
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.10
  Git commit:       70efc0e
  Built:            Tue Nov  3 21:41:31 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.4.2
  GitCommit:        b7a25261e029d68978bbf290df0a62fc819b3c51
 runc:
  Version:          1.5.2
  GitCommit:        v1.5.2-0-g29dd3dc2
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

2. Cas pràctic bàsic de comandos amb Docker.

El primer paso será comprobar y descargar la imagen oficial de Nginx

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker search --filter is-official=true nginx
NAME      DESCRIPTION                 STARS     OFFICIAL
nginx     Official build of Nginx.    21416     [OK]
```

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
a670537bed70: Pull complete
6fc242d221a6: Pull complete
a071113ac575: Pull complete
9f604dae09b2: Pull complete
6e2ab1f3ef4e: Pull complete
c139fe92686d: Pull complete
d322f1bf41d8: Pull complete
ca81da665177: Download complete
05a7d8870b4b: Download complete
Digest: sha256:5ab5b0b01097420873a3f3330f44248e34a3c2a3e8d5ee30658aeae8cb7435da
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Aquí podemos ver como se ha descargado la imagen correctamente

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest   5ab5b0b01097        243MB         66.8MB
```

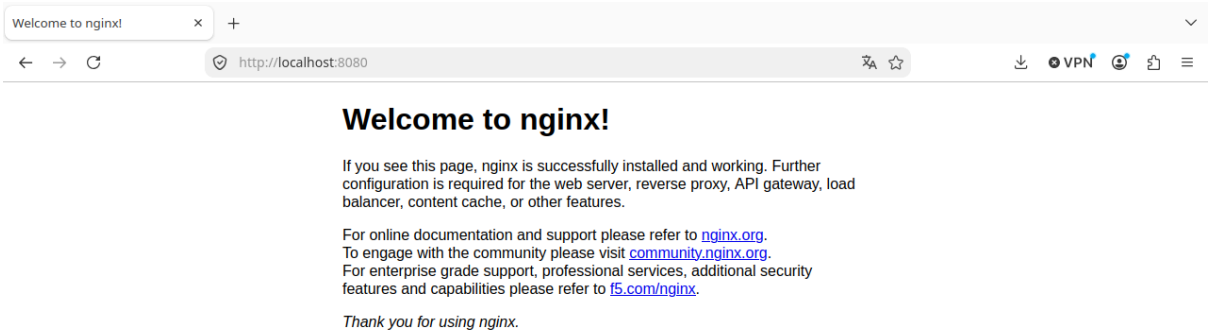
El siguiente paso será crear y arrancar el contenedor con el nombre servidor-practica y que sea accesible desde el puerto 8080

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker run -d --name servidor-practica -p 8080:80 nginx
0ebf951e505dbe2e66f33af8cc8fbe319e0f1a10622c5563315d311c8bc54de2
```

Con docker ps comprobaremos que está funcionando (aparece el contenedor servidor-practica en estado Up), además, se puede ver que conecta el puerto 8080 de nuestro ordenador con el puerto 80 del contenedor

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
        NAMES
0ebf951e505d   nginx     "/docker-entrypoint.…"   5 minutes ago   Up 5 minutes   0.0.0.0:8080->80/tcp, [::]:8080->
80/tcp   servidor-practica
```

Si accedemos a http://localhost:8000 podremos ver que funciona correctamente



Lo siguiente será detener el contenedor:

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker stop servidor-practica
servidor-practica
nacho@nacho-virtualbox:~/Desktop$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                     PORTS     NAMES
0ebf951e505d   nginx     "/docker-entrypoint.…"   10 minutes ago   Exited (0) 14 seconds ago             servidor-
practica
```

Y por último eliminarlo, si ejecutamos docker ps -a veremos que el contenedor servidor-practica ya no aparece

```
nacho@nacho-virtualbox:~/Desktop$ sudo docker rm servidor-practica
servidor-practica
nacho@nacho-virtualbox:~/Desktop$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
nacho@nacho-virtualbox:~/Desktop$
```

5. Cas pràctic de Docker Compose 1 de la unitat 6

El primer paso es crear la carpeta “wordpress-compose” donde crearemos posteriormente el fichero “docker-compose.yml“

```
nacho@nacho-virtualbox:~/Desktop$ sudo mkdir wordpress-compose
[sudo] password for nacho:
```

A continuación iremos a la carpeta creada anteriormente y crearemos el fichero mencionado con el contenido indicado en las transparencias, en este definiremos varios puntos de los servidores de bases de datos “db” y de los servidores con Apache + PHP + Wordpress “wordpress” (la versión del fichero no es necesaria indicarla ya que está obsoleta, esto será mencionado durante la puesta en marcha del propio sistema)

```
nacho@nacho-virtualbox:~/Desktop/wordpress-compose$ sudo nano docker-compse.yml
[sudo] password for nacho:
```

```
nacho@nacho-virtualbox: ~/Desktop/wordpress-compose
  GNU nano 7.2                        docker-compose.yml *
#Versión del fichero docker-compose 3.9. No obligatorio desde la versión de docker-co>
version: "3.9"
#Indicamos los servicios a lanzar
services:
#Plantill del servicio "db"
db:
#Se basa en la imagen "mariadb", version 10.11.2
image: mariadb:10.11.2
#Mapea en el volumen "db_data" el directorio "/var/lib/mysql", lo que da persistencia>
de
#Wordpress almacenado en la base de datos
volumes:
- db_data:/var/lib/mysql
#Indica que siempre que el servicio finalice, se reiniciará
restart: always
#Define un conjunto de variables de entorno para estos contenedores,
#indicando password de root de mariadb, nombre de base de datos,
# usuario con permisos root (necesario para conexiones remotas) y password de ese usu>
environment:
```

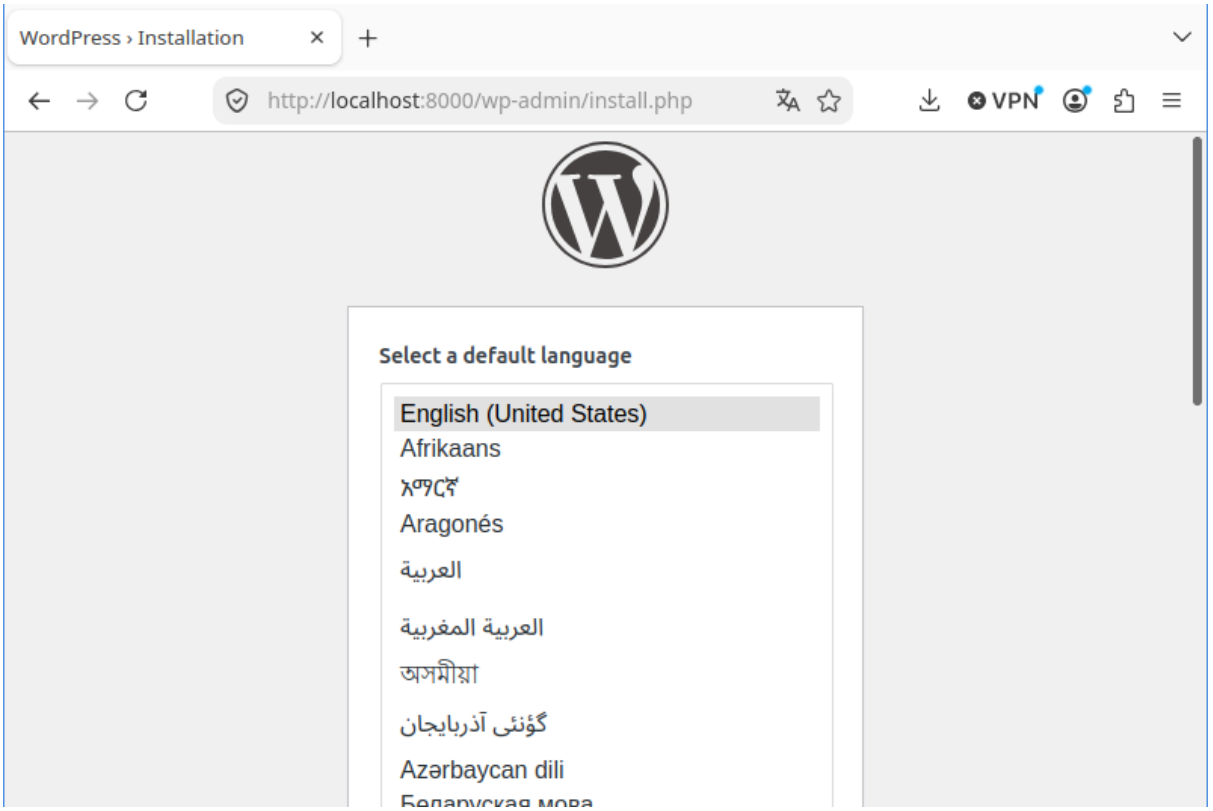
Para poner en marcha este sistema, iremos al directorio donde tenemos el fichero “docker-compose.yml” y ejecutamos el comando “docker compose up -d” (al poner en marcha el sistema podemos comprobar, tal como he mencionado antes, que la versión está obsoleta)

```
nacho@nacho-virtualbox:~/Desktop/wordpress-compose$ sudo docker compose up -d
WARN[0000] /home/nacho/Desktop/wordpress-compose/docker-compose.yml: the attribute `ve
rsion` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 40/40
 ✔ Image wordpress:latest                 Pulled                                45.4s
 ✔ Image mariadb:10.11.2                  Pulled                                35.9s
 ✔ Network wordpress-compose_default      Created                                0.2s
 ✔ Volume wordpress-compose_db_data       Created                                0.0s
 ✔ Container wordpress-compose-db-1       Started                                2.6s
 ✔ Container wordpress-compose-wordpress-1 Started                               3.6s
```

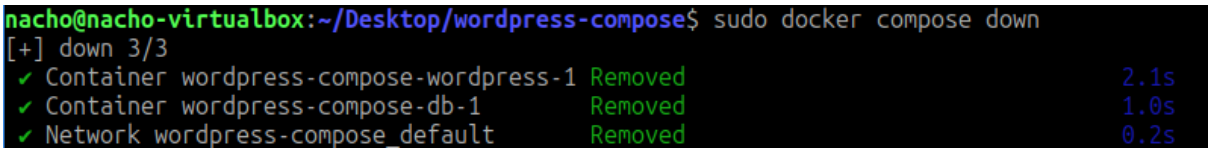
Para comprobar que todo se ha realizado correctamente podemos ejecutar el comando “docker compose ps”

```
nacho@nacho-virtualbox:~/Desktop/wordpress-compose$ sudo docker compose ps
NAME                            IMAGE              COMMAND                  SERVICE
  CREATED         STATUS          PORTS
wordpress-compose-db-1          mariadb:10.11.2    "docker-entrypoint.s…"   db
  2 minutes ago   Up 2 minutes    3306/tcp
wordpress-compose-wordpress-1   wordpress:latest   "docker-entrypoint.s…"   wordpress
  2 minutes ago   Up 2 minutes    0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
```

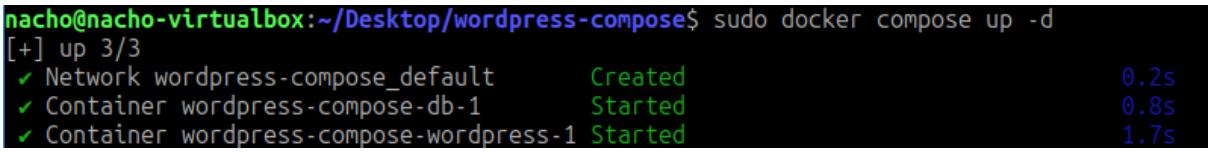
Si accedemos a http://localhost:8000 podremos ver algo similar a esto:



Para realizar una parada del sistema ejecutaremos el comando “docker compose down”



Si hacemos un relanzado del sistema con “docker compose up -d” podemos ver que el sistema aprovecha las imágenes ya creadas



ACTIVITAT 03

1. ¿Qué diferencia principal hay entre imagen y contenedor?

- Imagen: Es una plantilla que contiene todo lo necesario para ejecutar una aplicación (programa, dependencias y configuración).
- Contenedor: Es una instancia creada a partir de una imagen que permite ejecutar esa aplicación.

2. ¿Qué comando arranca un contenedor a partir de una imagen?

docker run nombre_imagen

3. ¿Por qué un contenedor pesa mucho menos que una máquina virtual?

Porque un contenedor comparte el núcleo (kernel) del sistema operativo del ordenador anfitrión, mientras que una máquina virtual necesita un sistema operativo completo.
Por eso, los contenedores ocupan menos espacio, consumen menos recursos y se inician más rápido.

4. Añade 4 comandos diferentes y explica qué hacen con un ejemplo para cada uno.

1. docker ps
Muestra los contenedores que están en ejecución.

2. docker images
Muestra las imágenes que tenemos descargadas en Docker.

3. docker stop
Detiene un contenedor que está en ejecución.

4. docker rm
Elimina un contenedor que ya está detenido.